

3. Python Solution

```
from typing import List

class Solution:
    def solveSudoku(self, board: List[List[str]]) -> None:
        # Bitmask per row/col/box: bit (d-1) set means digit d is already used.
        # Integers + bitwise ops give O(1) constraint checks with tiny memory.
        rows = [0] * 9
        cols = [0] * 9
        boxes = [0] * 9 # box index = (r // 3) * 3 + (c // 3)

        # Collect empty cells once so recursion just walks an index forward.
        empties = []

        # Seed the masks from the givens before searching.
        for r in range(9):
            for c in range(9):
                ch = board[r][c]
                if ch == '.':
                    empties.append((r, c))
                else:
                    bit = 1 << (int(ch) - 1)
                    rows[r] |= bit
                    cols[c] |= bit
                    boxes[(r // 3) * 3 + (c // 3)] |= bit

    def backtrack(i: int) -> bool:
        # All blanks filled -> a complete, valid board.
        if i == len(empties):
            return True

        r, c = empties[i]
        b = (r // 3) * 3 + (c // 3)
        # Any bit set in any of the three masks is forbidden here.
        used = rows[r] | cols[c] | boxes[b]

        for d in range(9):
            bit = 1 << d
            if used & bit:
                continue # digit d+1 already present in row/col/box

            # Choose: place digit and mark it in all three masks.
            board[r][c] = str(d + 1)
            rows[r] |= bit
            cols[c] |= bit
            boxes[b] |= bit

            if backtrack(i + 1):
                return True

            # Un-choose: clear the bit (XOR) and restore the blank.
            rows[r] ^= bit
            cols[c] ^= bit
            boxes[b] ^= bit
            board[r][c] = '.'

        # No digit fit -> force caller to try a different earlier choice.
        return False

    backtrack(0)
```

4. Complexity Analysis

Time: $O(9^m)$ in the worst case, where m is the number of empty cells (≤ 81). Each blank branches into up to 9 digits. This is a loose upper bound — the constraint masks prune the vast majority of branches immediately, so real puzzles solve in a tiny fraction of that. Because the board is fixed at 9×9 , this is technically $O(1)$, but the exponential form better describes the search effort.

Space: $O(m)$ for the recursion stack (depth = number of blanks). The three mask arrays and the `empties` list are $O(81) = O(1)$. Using integer bitmasks instead of `set` objects keeps the per-frame overhead minimal.

5. Alternative Approaches

- **Sets instead of bitmasks.** Track used digits with `set()` per row/col/box. Same Big-O and arguably more readable, but bit operations are faster and lighter in practice.
- **Most-Constrained-Variable (MRV) heuristic.** Instead of filling blanks left-to-right, always pick the empty cell with the fewest legal candidates next. This slashes the branching factor and dramatically speeds up hard/sparse puzzles, at the cost of a scan to pick the next cell.
- **Dancing Links (Algorithm X).** Model Sudoku as an exact-cover problem and solve with Knuth's DLX. The fastest known general approach, but far more code and overkill for a single 9×9 grid.

Codex Solution

Solution not available